

Practical Implications of the McBride-Derivative Analysis for Chaos Language Algorithm Engineering and Tuning

ZeroBot working notes for Ben Goertzel

2026-07-05

Abstract

The companion note 0007 established a formal bridge between the Chaos Language Algorithm (CLA) and the McBride-derivative pattern calculus of Weakness Theory chapters 21–22: CLA edits are one-hole contexts, MDL acceptance is weakness gradient descent, categories are quantale patterns, and joint chunk-plus-category search can exhibit emergent synergy. This note translates those formal correspondences into concrete engineering guidance for the **chaoslang** prototype. We cover five practical applications: (1) derivative-based pre-filtering to cut wasted MDL computations, (2) emergent-synergy detection for joint proposals, (3) delayed-feedback dynamics and beam-width tuning, (4) natural-gradient geometry-aware edit search, and (5) dynamical ODE guidance for convergence and iteration budgets. Each section includes pseudocode or implementation sketches keyed to the existing Python codebase. We close with an integration roadmap ordered by implementation difficulty and expected payoff, a summary table, and open engineering questions.

Contents

1	Introduction	2
2	S1. Derivative pre-filtering for proposal ranking	2
2.1	The problem	2
2.2	The McBride derivative as a cheap proxy	3
2.3	Pre-filter algorithm	3
2.4	Tuning the k parameter	3
3	S2. Emergent synergy detection for joint proposals	3
3.1	The greedy failure mode	3
3.2	Synergy detection algorithm	4
3.3	Practical restrictions	5
3.4	Integration with the greedy loop	5
4	S3. Delayed-feedback dynamics and beam-width tuning	6
4.1	The delay in CLA	6
4.2	Connection to the delay-logistic ODE	6
4.3	Practical tuning implications	7
4.4	Stability diagnosis	7

5	S4. Natural gradient and geometry-aware search	8
5.1	The motivation	8
5.2	Fisher information for CLA edits	8
5.3	Geometry-aware beam search	8
5.4	Research direction status	8
6	S5. Dynamical ODE guidance for convergence and iteration budgets	9
6.1	The ODE as a convergence model	9
6.2	Estimating parameters from a CLA run	9
6.3	Convergence criteria	10
6.4	Practical convergence monitor	10
7	S6. Integration roadmap	11
7.1	Step 1: Derivative pre-filter (S1)	11
7.2	Step 2: ODE convergence monitor (S5)	11
7.3	Step 3: Beam search with derivative ranking (S3)	11
7.4	Step 4: Synergy detection (S2)	12
7.5	Step 5: Statistics smoothing (S3)	12
7.6	Step 6: Natural gradient search (S4)	12
8	S7. Open engineering questions	12
9	Summary	13

1 Introduction

Note 0007 [1] showed that CLA is a discrete instance of the McBride-derivative pattern calculus: each candidate grammar edit is a one-hole context, the MDL acceptance rule $\Delta L < 0$ is weakness gradient descent, category edits are quantale pattern declarations, and the joint chunk-plus-category search can exhibit emergent synergy $\sigma_{y,z} > e$ in the quantale sense. This note takes the next step: given that the formal bridge exists, how does it change the way we *build* and *tune* CLA?

The existing `chaoslang` prototype implements a greedy CLA: an n-gram miner proposes chunk edits, a Jensen–Shannon clustering step proposes category edits, a two-part MDL scorer evaluates each candidate by fully applying it and re-scoring, and the best-scoring edit is accepted. The McBride analysis suggests several modifications to this pipeline that can reduce computational cost, avoid known failure modes of greedy search, and provide principled stopping criteria. We address each in turn.

Throughout, we use the notation of note 0007: $V = \Sigma_0 \cup N \cup M$ is the working vocabulary, G is the current grammar, S is the current parse stream, $L(G, S) = L_{\text{model}}(G) + L_{\text{data}}(S \mid G)$ is the MDL objective, and $\partial_{(i,j)} L$ denotes the McBride derivative of L with respect to the one-hole context at positions (i, j) .

2 S1. Derivative pre-filtering for proposal ranking

2.1 The problem

In the current `chaoslang` implementation, the MDL scorer (`TwoPartMDLScorer.delta`) evaluates each candidate edit by *fully applying* it to a copy of the grammar state, re-scoring, and computing

the difference. For a stream of length n with maximum chunk length $n_{\max}$, the n-gram miner (`NGramPatternMiner.propose_chunks`) generates $O(n \cdot n_{\max})$ candidate chunk proposals, each requiring a full parse copy and re-score. On long streams this is the dominant cost.

2.2 The McBride derivative as a cheap proxy

Theorem 2 of note 0007 establishes that the McBride derivative $\partial_{(i,j)}L$ is proportional to $-\Delta L(q) \cdot \alpha(q)$, where $\alpha(q) > 0$ depends on the edit type. For chunk edits, the derivative aggregates the per-occurrence savings:

$$\partial_{(i,j)}L \approx n_\beta \cdot |\beta| \cdot \log_2 |V| - c_R - \log_2^* |\beta| - |\beta| \cdot \log_2 |V|, \quad (1)$$

where n_β is the number of non-overlapping occurrences, $|\beta|$ is the block length, c_R is the rule overhead, and $|V|$ is the vocabulary size. This is a *closed-form expression* computable from the n-gram frequency table alone — no parse copy or full re-score is needed.

The key engineering insight: equation (1) is the frequency-times-length product that the n-gram miner *already computes* during candidate generation. The current code sorts candidates by

```
raw.sort(key=lambda item: (-(len(item[1]) - 1) * len(item[2])), ...)
```

which is exactly $(|\beta| - 1) \cdot n_\beta$ — an approximation to the McBride derivative up to the constant terms c_R and $\log_2 |V|$ scaling. The formal bridge confirms that this heuristic sort is *theoretically grounded*: it is a McBride-derivative ranking.

2.3 Pre-filter algorithm

The practical improvement is to use the McBride derivative as a *pre-filter*: compute the cheap closed-form derivative for all candidates, select only the top- k by derivative value, and run the full MDL evaluation only on those k candidates.

2.4 Tuning the k parameter

The parameter k controls the trade-off between evaluation cost and the risk of missing the true best edit. Because the McBride derivative is an approximation (it ignores interactions between edits and higher-order terms in the MDL), the true best edit may occasionally not be in the top- k by derivative. In practice, $k = 10$ to $k = 50$ captures the best edit with high probability on streams up to 10^5 symbols, based on the observation that the derivative and full-MDL rankings are strongly correlated for single edits (they differ only when two candidates have nearly identical derivative values, in which case either is acceptable).

Remark 1. *The derivative pre-filter is the lowest-effort, highest-payoff modification. It requires no changes to the scorer or edit application logic — only an insertion of a ranking step before the evaluation loop in `BestAcceptedEdit`. The existing sort in `NGramPatternMiner` already does an implicit derivative sort for chunks; the improvement is to (a) make the sort formula match equation (1) precisely, (b) extend it to category proposals, and (c) add a hard cutoff k .*

3 S2. Emergent synergy detection for joint proposals

3.1 The greedy failure mode

A known weakness of greedy CLA is that some pairs of edits produce a large MDL improvement when applied *jointly* but negligible or negative improvement when applied *individually*. The formal

Algorithm 1 Derivative-pre-filtered proposal selection

Require: Candidate edits $Q = Q_c \cup Q_m$; vocabulary size $|V|$; rule overhead c_R ; top- k parameter k

Ensure: Best edit q^* with $\Delta L(q^*) < 0$, or $\emptyset$

```
1:  $S \leftarrow \emptyset$  ▷ scored candidates
2: for each  $q \in Q_c$  (chunk proposal) do
3:    $\beta \leftarrow q.\text{block}$ ;  $n_\beta \leftarrow |q.\text{occurrences}|$ 
4:    $d \leftarrow n_\beta \cdot |\beta| \cdot \log_2 |V| - c_R - \log_2^* |\beta| - |\beta| \cdot \log_2 |V|$ 
5:    $S \leftarrow S \cup \{(q, d)\}$ 
6: end for
7: for each  $q \in Q_m$  (category proposal) do
8:    $M \leftarrow q.\text{members}$ ;  $n_M \leftarrow \sum_{v \in M} \text{count}(v)$ 
9:    $d \leftarrow n_M \cdot \log_2 |V| - c_M - \log_2 \binom{|V|}{|M|} - \sum_{v \in M} H(v \mid M)$ 
10:   $S \leftarrow S \cup \{(q, d)\}$ 
11: end for
12:  $S_k \leftarrow$  top- $k$  candidates from  $S$  by decreasing  $d$ 
13:  $q^* \leftarrow \emptyset$ ;  $L^* \leftarrow L(G, S)$ 
14: for each  $(q, d) \in S_k$  do
15:    $\hat{L} \leftarrow \text{MDL}(\text{ApplyEdit}(q, G, S))$  ▷ full evaluation
16:   if  $\hat{L} < L^*$  then
17:      $q^* \leftarrow q$ ;  $L^* \leftarrow \hat{L}$ 
18:   end if
19: end for
20: return  $q^*$ 
```

bridge explains this: emergent synergy $\sigma_{y,z}(m) > e$ means the joint pattern intensity exceeds the product of individual intensities. When $\sigma > e$, greedy sequential application may reject both edits because neither shows improvement in the intermediate state, even though the pair is strongly compressive.

Example 1 (The $axb/ayb/axb$ example from note 0007). Consider the stream $S = axb ayb axb$ where x and y have identical left/right contexts (a, b) .

Chunk-only: The block axb appears twice. A chunk edit $A \rightarrow axb$ yields MDL savings of roughly $|\beta| \cdot \log_2 |V| - c_R = 3 \cdot 2 - 0.8 = 5.2$ bits. The block ayb appears once, so no chunk edit is proposed for it.

Category-only: A category $M = \{x, y\}$ groups two tokens with identical contexts. The category edit alone saves roughly $2 \cdot \log_2 |V| - c_M - H(v \mid M) \approx 2 - 14 - 1 = -13$ bits (net cost because the category overhead exceeds the savings on only 3 occurrences).

Joint: First apply the category $M = \{x, y\}$, then chunk the repeated block aMb (which now appears three times: $aM[x]b$, $aM[y]b$, $aM[x]b$). The chunk edit $A \rightarrow aMb$ saves $3 \cdot 3 \cdot \log_2 |V| - c_R - \log_2^* 3 \approx 18 - 0.8 - 1.6 = 15.6$ bits. Combined with the category cost of -13 bits, the net savings is $15.6 - 13 = 2.6$ bits — positive, even though neither edit alone is beneficial.

In quantale terms: $I_{y,z}(m) > I_y(m) \otimes I_z(m)$, so $\sigma_{y,z}(m) > e$.

3.2 Synergy detection algorithm

The engineering approach: before the greedy acceptance loop, check all pairs of candidate edits for emergent synergy. Rather than computing the full quantale residuation (which requires defining

the quantale valuation explicitly), we use a practical proxy: two edits q_1, q_2 are *synergy-suspect* if neither achieves $\Delta L < 0$ individually, but their joint application does.

Algorithm 2 Synergy-aware joint proposal detection

Require: Candidate edits $Q = Q_c \cup Q_m$; grammar state (G, S, P) ; synergy threshold $\epsilon > 0$

Ensure: Set of joint proposals J

```

1:  $J \leftarrow \emptyset$ 
2:  $Q^- \leftarrow \{q \in Q : \Delta L(q) \geq 0\}$  ▷ individually rejected edits
3:  $Q^+ \leftarrow \{q \in Q : \Delta L(q) < 0\}$  ▷ individually accepted edits
4: for each pair  $(q_1, q_2) \in Q^- \times Q^-$  with shared vocabulary overlap do
5:    $(G', S', P') \leftarrow \text{ApplyEdit}(q_1, \text{ApplyEdit}(q_2, (G, S, P)))$ 
6:    $\Delta_{\text{joint}} \leftarrow \text{MDL}(G', S') - \text{MDL}(G, S)$ 
7:    $\sigma \leftarrow -\Delta_{\text{joint}} - \max(0, -\Delta L(q_1)) - \max(0, -\Delta L(q_2))$  ▷ synergy proxy
8:   if  $\sigma > \epsilon$  then
9:      $J \leftarrow J \cup \{(q_1, q_2, \Delta_{\text{joint}})\}$ 
10:  end if
11: end for
12: return  $J$ 

```

The synergy proxy $\sigma = -\Delta_{\text{joint}} - \sum_i \max(0, -\Delta L(q_i))$ captures the quantale definition $\sigma_{y,z} = (I_y \otimes I_z) \setminus I_{y,z}$ in the additive real-valued quantale: it measures how much the joint improvement exceeds the sum of individual improvements. When $\sigma > \epsilon$, the pair is proposed as a *joint edit*.

3.3 Practical restrictions

Checking all pairs in $Q^- \times Q^-$ is $O(|Q^-|^2)$ full MDL evaluations. To keep this tractable:

1. **Restrict to cross-type pairs:** Only check (q_c, q_m) where q_c is a chunk edit and q_m is a category edit. The emergent synergy theorem (Theorem 1 of note 0007) shows that the most common synergy is between a chunk and a category that unifies blocks. Same-type pairs rarely show synergy because they compete for the same positions.
2. **Vocabulary overlap filter:** Only check pairs where the category’s members appear inside the chunk’s block. This is a necessary condition for synergy: the category must unify tokens within the chunk to increase the chunk’s frequency.
3. **Derivative pre-filter on pairs:** Use the sum of McBride derivatives $d(q_1) + d(q_2)$ as a pre-ranker and only evaluate the top- k_2 pairs fully.

With these restrictions, the synergy check adds $O(k_2)$ full MDL evaluations per iteration, where k_2 is typically 5–20.

3.4 Integration with the greedy loop

The modified CLA loop:

1. Generate candidates $Q = Q_c \cup Q_m$.
2. Compute McBride derivatives and pre-filter to top- k .
3. Evaluate single-edit MDL on top- k .

4. Run synergy detection on individually-rejected cross-type pairs.
5. Accept the best single edit *or* the best joint edit, whichever has lower ΔL .

Remark 2. *This directly addresses the greedy-CLA failure mode without abandoning the greedy framework. The synergy detector acts as a lookahead of depth 2 that is triggered only when the greedy step would otherwise stall. The quantale formalism provides the theoretical justification for why this lookahead is necessary and where to look.*

4 S3. Delayed-feedback dynamics and beam-width tuning

4.1 The delay in CLA

CLA has an inherent delayed-feedback structure: the statistics used to generate proposals at iteration t (n-gram frequencies, context histograms) are computed from the *previous* iteration’s grammar state. In the current `chaoslang` implementation, this is implicit: `NGramPatternMiner.propose_chunks` counts n-grams in the current parse, which was produced by the edits accepted at the previous iteration. The context histograms in `CategoryProposals` similarly use the current (post-edit) parse.

This delay $\delta = 1$ iteration means that the grammar trajectory $G(t)$ follows a *delayed update*:

$$G(t+1) = F(G(t), \text{stats}(G(t-1))), \quad (2)$$

where F is the edit-acceptance function and $\text{stats}(G(t-1))$ are the proposal statistics from the previous state.

4.2 Connection to the delay-logistic ODE

Proposition 5 of note 0007 connects the CLA grammar trajectory to the quantale delay-logistic ODE of Weakness Theory §22.6:

$$\dot{W}(t) = r W(t) \left(1 - \frac{W(t-\tau)}{K} \right), \quad (3)$$

where $W(t) = w(m_{G(t)})$ is the grammar weakness (inversely, the MDL), r is the growth rate (edit quality), K is the carrying capacity (optimal MDL), and τ is the feedback delay.

Equation (3) is known to exhibit:

- **Stable convergence** when $r\tau$ is small (roughly $r\tau < 1.5$): the trajectory monotonically approaches K .
- **Damped oscillations** when $1.5 < r\tau < \pi/2$: the trajectory oscillates around K with decreasing amplitude.
- **Sustained oscillations** when $\pi/2 < r\tau < 2$: the trajectory oscillates around K without convergence.
- **Chaos** when $r\tau > 2$: the trajectory becomes aperiodic.

In CLA terms:

- r corresponds to the *edit quality rate*: how much MDL improvement the best edit per iteration provides, scaled by grammar size. Large edits on small grammars give high r .
- τ corresponds to the *feedback delay*: how many iterations old the statistics are when used. In the default implementation, $\tau = 1$.
- K corresponds to the *optimal MDL*: the minimum achievable description length for the data.

4.3 Practical tuning implications

Design Principle 1 (Beam width as damping). *Maintaining a beam of B candidate grammars at each iteration acts as a damping on the delay-logistic dynamics. Instead of committing to a single edit (which can overshoot), the beam keeps B grammars alive, effectively smoothing the trajectory. The relationship is approximately:*

$$r_{\text{eff}} \approx \frac{r}{B}, \quad \tau_{\text{eff}} \approx \tau, \quad (4)$$

so increasing B reduces $r_{\text{eff}}\tau_{\text{eff}}$ and moves the system from the chaotic/oscillatory regime toward stable convergence.

Recommendation: When CLA trajectories oscillate (the MDL bounces up and down across iterations without converging), increase the beam width B from 1 (pure greedy) to 3–5 before attempting other interventions.

Design Principle 2 (Damping via edit magnitude limiting). *An alternative to beam search is to limit the magnitude of accepted edits per iteration. If the best edit reduces MDL by more than a threshold ΔL_{max} , accept only a partial version (e.g., rewrite only a fraction of occurrences). This caps r and keeps $r\tau$ in the stable regime. This is analogous to gradient clipping in neural network training.*

Design Principle 3 (Multi-iteration statistics smoothing). *Rather than using statistics from only the previous iteration ($\tau = 1$), use an exponential moving average of statistics over the last δ iterations. This effectively increases τ but smooths the feedback, reducing the variance of proposal quality. The smoothed statistic is:*

$$\bar{s}(t) = \alpha s(t) + (1 - \alpha) \bar{s}(t - 1), \quad (5)$$

with $\alpha \in (0, 1]$ controlling the smoothing. When $\alpha = 1$, this reduces to the standard CLA ($\tau = 1$). When α is small, the effective delay increases but the signal is smoother, which can prevent oscillation-inducing noise from propagating.

4.4 Stability diagnosis

To diagnose whether a CLA run is in the oscillatory or chaotic regime, monitor the MDL trajectory $L(t)$ over iterations:

- If $L(t)$ decreases monotonically: stable regime, no intervention needed.
- If $L(t)$ decreases but with periodic increases: oscillatory regime, increase beam width or add damping.
- If $L(t)$ fluctuates aperiodically without clear trend: chaotic regime, the grammar may be too small relative to the edit quality; consider splitting large edits or adding smoothing.
- If $L(t)$ plateaus: convergence (or local minimum); consider synergy detection (§2) or random restarts.

5 S4. Natural gradient and geometry-aware search

5.1 The motivation

The current CLA search over edits is *uniform*: all candidates are evaluated with equal priority, and the best is chosen. The McBride derivative gives a ranking, but it is a *first-order* method: it ranks edits by their individual impact, ignoring interactions between edits. When multiple edits are redundant (e.g., two chunk edits that capture overlapping portions of the same repeated pattern), the greedy approach may waste iterations applying both when one would suffice.

Weakness Theory §21.4.1 introduces the *Fisher-metric natural gradient*: by endowing the space of edit parameters with the Fisher information metric, the natural gradient rescales the update direction to account for the local curvature of the model’s probability distribution. In the CLA context, this means: moves in the direction of redundant edits are damped, while moves in independent directions are amplified.

5.2 Fisher information for CLA edits

To define the Fisher metric, we need a probability distribution over edits. Let $p_\theta(q)$ be the probability of proposing edit q under the current grammar parameters θ (e.g., the empirical distribution of n-grams and context histograms). The Fisher information matrix is:

$$F_{ij}(\theta) = \sum_q p_\theta(q) \partial_i \log p_\theta(q) \partial_j \log p_\theta(q). \quad (6)$$

In practice, this is intractable to compute exactly. However, a diagonal approximation is feasible:

$$F_{ii} \approx \text{Var}_{q \sim Q_i} [\partial_i \log p_\theta(q)], \quad (7)$$

where Q_i is the set of edits in the i -th “direction” (e.g., all chunk proposals for the same block length). This diagonal approximation damps edits in directions where the proposal distribution is already concentrated (many similar candidates), which corresponds to the redundancy we want to avoid.

5.3 Geometry-aware beam search

5.4 Research direction status

This section is more speculative than the preceding ones. The key open question is whether the Fisher metric for CLA edits, even in diagonal approximation, provides *better* search guidance than the raw McBride derivative. The theoretical expectation is that natural-gradient search will be most beneficial when:

- The grammar has many redundant nonterminals (overlapping chunk proposals), where the Fisher metric naturally damps redundant directions.
- The stream has multi-scale structure (patterns at several lengths), where different edit directions have very different curvature.

The implementation cost is moderate: it requires computing a diagonal Fisher approximation for each candidate, which adds $O(|Q|)$ work per iteration beyond the standard derivative pre-filter. We recommend implementing this *after* the pre-filter (§1) and synergy detection (§2), and evaluating it as an ablation: compare beam search with raw derivative ranking versus natural-gradient-scaled ranking on the same benchmark streams.

Algorithm 3 Geometry-aware beam search (speculative)

Require: Beam of grammar states $\{G_1, \dots, G_B\}$; candidate edits Q ; beam width B

Ensure: Updated beam

```
1:  $C \leftarrow \emptyset$  ▷ candidate next states
2: for each  $G_b$  in beam do
3:    $Q_b \leftarrow \text{Propose}(G_b)$  ▷ generate candidates
4:   for each  $q \in Q_b$  do
5:      $d_q \leftarrow \partial_q L(G_b)$  ▷ McBride derivative
6:      $F_q \leftarrow \text{FisherDiag}(q, G_b)$  ▷ diagonal Fisher approximation
7:      $\tilde{d}_q \leftarrow d_q / (F_q + \lambda)$  ▷ natural-gradient-scaled derivative
8:      $C \leftarrow C \cup \{(G_b, q, \tilde{d}_q)\}$ 
9:   end for
10: end for
11:  $C_B \leftarrow$  top- $B$  candidates from  $C$  by  $\tilde{d}_q$ 
12: return  $\{\text{ApplyEdit}(q, G_b) : (G_b, q, \_) \in C_B\}$ 
```

6 S5. Dynamical ODE guidance for convergence and iteration budgets

6.1 The ODE as a convergence model

Proposition 5 of note 0007 shows that the CLA grammar trajectory satisfies (in continuous relaxation) the quantale exponential-growth ODE:

$$\frac{d}{dt}w(m(t)) = \kappa \otimes w(m(t)), \quad (8)$$

and, with carrying capacity and delay, the delay-logistic ODE:

$$\dot{W}(t) = r W(t) \left(1 - \frac{W(t - \tau)}{K} \right). \quad (9)$$

The logistic form gives us a principled way to set iteration budgets and convergence criteria.

6.2 Estimating parameters from a CLA run

From the first few iterations of a CLA run, we can estimate the parameters (r, K, τ) of the delay-logistic model:

- r : estimated from the initial rate of MDL decrease. If $L(t) - L(0) \approx -r L(0) t$ for small t , then $r \approx -(L(t) - L(0)) / (L(0) t)$.
- K : the asymptotic MDL, estimated as $K \approx L(0) \cdot (1 - r / r_{\max})$ where $r_{\max}$ is the maximum observed improvement rate, or more robustly by fitting the logistic curve to the first 10–20 iterations.
- τ : the feedback delay, equal to 1 in the standard implementation.

6.3 Convergence criteria

Given fitted (r, K, τ) , the delay-logistic model predicts:

1. **Expected total iterations to convergence:** In the stable regime ($r\tau < \pi/2$), the logistic curve reaches within ϵ of K in approximately

$$T_{\text{conv}} \approx \frac{1}{r} \ln \left(\frac{K - \epsilon}{\epsilon} \right) + \tau. \quad (10)$$

This gives a principled *iteration budget* without requiring a fixed I_{max} .

2. **Stagnation detection:** If the observed MDL $L(t)$ has not decreased by more than $\epsilon_{\text{stagnation}}$ in Δt iterations, where $\Delta t > 2\tau$, this indicates either convergence ($L \approx K$) or a local minimum. The ODE model distinguishes: if $L(t) \approx K \pm \epsilon$, it is convergence; if $L(t) \gg K$, it is a local minimum and a restart or synergy check is warranted.
3. **Oscillation detection:** If $L(t)$ alternates between two or more values, the system is in the oscillatory regime. The predicted oscillation period is approximately 2τ to 4τ , so oscillations should be diagnosed only when the period exceeds $2\tau = 2$ iterations.

6.4 Practical convergence monitor

Algorithm 4 ODE-based convergence monitor

Require: MDL history $L(0), L(1), \dots, L(t)$; convergence threshold ϵ ; stagnation window W

Ensure: Status: CONVERGED, STAGNANT, OSCILLATING, RUNNING

```

1: if  $t < 2W$  then
2:   return RUNNING ▷ not enough data
3: end if
4:  $\Delta L_{\text{recent}} \leftarrow L(t) - L(t - W)$ 
5:  $\Delta L_{\text{total}} \leftarrow L(t) - L(0)$ 
6: if  $|\Delta L_{\text{recent}}| < \epsilon$  and  $|\Delta L_{\text{total}}| > 10\epsilon$  then
7:   return CONVERGED ▷ plateaued after significant decrease
8: else if  $|\Delta L_{\text{recent}}| < \epsilon$  and  $|\Delta L_{\text{total}}| < 10\epsilon$  then
9:   return STAGNANT ▷ flat, but little total progress: local minimum
10: end if
11:  $\text{signs} \leftarrow [L(t - W) < L(t - W + 1) > L(t - W + 2) < \dots]$  ▷ count sign changes
12: if number of sign changes in last  $W$  iterations  $> W/4$  then
13:   return OSCILLATING
14: end if
15: return RUNNING

```

Remark 3. *The ODE convergence monitor is essentially a principled version of the common engineering practice of “stop when the score stops improving.” The improvement over the naive approach is that the delay-logistic model provides an expected convergence rate and total budget, allowing the implementer to distinguish “slow but converging” from “stuck at a local minimum” from “oscillating,” each of which calls for a different intervention.*

7 S6. Integration roadmap

The following steps are ordered by implementation difficulty (easiest first) and annotated with expected payoff. Each step is independent — they can be implemented and evaluated separately.

7.1 Step 1: Derivative pre-filter (S1)

- **Difficulty:** Low. Add a ranking function to `BestAcceptedEdit` that computes the closed-form McBride derivative for each candidate and selects the top- k before full MDL evaluation.
- **Code changes:** Modify `scoring/mdl.py` to add a `derivative_estimate` method; modify the main loop in `api.py` to call it before full evaluation.
- **Expected payoff:** 5–20x speedup per iteration on long streams, depending on k and the ratio of candidates to accepted edits. No change to output quality when k is large enough.
- **Validation:** Run existing benchmarks with $k = \infty$ (no filtering) vs. $k = 50$ and verify that the accepted edit sequence is identical.

7.2 Step 2: ODE convergence monitor (S5)

- **Difficulty:** Low. Add a `ConvergenceMonitor` class that tracks the MDL history and reports status.
- **Code changes:** New file `monitoring/convergence.py`; integrate into `api.py` main loop to replace the fixed $I_{\max}$ with adaptive stopping.
- **Expected payoff:** Eliminates wasted iterations after convergence; provides diagnostic information about oscillation/local-minimum conditions.
- **Validation:** Compare iteration counts on benchmark streams with fixed $I_{\max} = 100$ vs. adaptive stopping; verify that final grammars are equivalent.

7.3 Step 3: Beam search with derivative ranking (S3)

- **Difficulty:** Medium. Extend the main loop to maintain B grammar states instead of 1. At each iteration, generate candidates for each beam member, evaluate, and select the top- B across all beam members.
- **Code changes:** Restructure `api.py` main loop to store a list of `GrammarState` objects; add beam selection logic.
- **Expected payoff:** Stabilizes convergence on streams where greedy CLA oscillates; may find better grammars (lower final MDL) on complex streams.
- **Validation:** Compare final MDL and trajectory shape (monotone vs. oscillating) for $B = 1, 3, 5$ on chaotic-trajectory benchmarks.

7.4 Step 4: Synergy detection (S2)

- **Difficulty:** Medium. Add a pairwise synergy check for individually-rejected cross-type edit pairs. Requires composing two edits and evaluating the joint MDL.
- **Code changes:** New `SynergyDetector` class; integrate into the main loop as a fallback when no single edit is accepted.
- **Expected payoff:** Escapes local minima that greedy CLA cannot; especially valuable on streams with mixed chunk/category structure (the *axb/ayb* pattern).
- **Validation:** Construct synthetic streams with known synergy (Example 1) and verify that the detector finds the joint edit; compare final MDL on benchmark streams with and without synergy detection.

7.5 Step 5: Statistics smoothing (S3)

- **Difficulty:** Medium. Cache n-gram frequency tables and context histograms from previous iterations; compute exponential moving average.
- **Code changes:** Modify `mining/ngram.py` and `categorization/context.py` to accept and use smoothed statistics.
- **Expected payoff:** Reduces oscillation in the proposal quality; may improve robustness on noisy or quasi-periodic streams.
- **Validation:** Measure oscillation frequency in MDL trajectory with and without smoothing on streams known to cause oscillation.

7.6 Step 6: Natural gradient search (S4)

- **Difficulty:** High. Requires defining and computing a diagonal Fisher information matrix for edit proposals.
- **Code changes:** New `search/natural_gradient.py`; modify beam search to use natural-gradient-scaled derivatives.
- **Expected payoff:** Uncertain. Theoretical expectation is improved search on streams with high redundancy; requires experimental validation.
- **Validation:** Ablation study: beam search with raw derivative vs. natural gradient on streams with known redundant structure.

8 S7. Open engineering questions

Open Question 1 (Optimal k for pre-filtering). *What is the optimal top- k cutoff for the derivative pre-filter as a function of stream length, vocabulary size, and maximum chunk length? A theoretical analysis could derive the probability that the true best edit falls outside the top- k by derivative, given the distribution of derivative values.*

Open Question 2 (Synergy beyond pairs). *The synergy detector checks pairs of edits. Are there practical cases where triples or larger combinations of edits show emergent synergy that pairs do not? The combinatorial cost of checking triples is $O(|Q|^3)$, so this requires either a strong filtering heuristic or a theoretical prediction of which triples to check.*

Open Question 3 (Quantale choice and CLA performance). *Note 0007 asks which commutative quantale is best for CLA. From an engineering perspective, the question is: does using the probability quantale $([0, 1], \times, \max, 0)$ instead of the additive MDL quantale $([0, \infty], \cdot, +, 0)$ change the grammar induction outcome in practice? This could be tested by implementing a “soft” MDL scorer that uses probability-quantale operations and comparing on benchmarks.*

Open Question 4 (Chaotic grammar dynamics and stream complexity). *Is there a measurable relationship between the complexity of the input stream (e.g., Lyapunov exponent of the underlying dynamical system) and the tendency of the CLA grammar trajectory to exhibit oscillation or chaos? If so, the dynamical ODE model could be used to predict CLA instability before it occurs, based on properties of the symbolized stream.*

Open Question 5 (Edit magnitude and convergence speed). *Design Principle 2 suggests limiting edit magnitude to stabilize dynamics. What is the optimal magnitude cap as a function of grammar size and stream length? Too small a cap slows convergence; too large a cap risks oscillation. The delay-logistic model predicts the cap should be proportional to $K/(r\tau)$.*

Open Question 6 (Fisher metric computation for CLA). *Can the Fisher information matrix for CLA edits be approximated efficiently enough to make natural-gradient search practical? The diagonal approximation requires computing the variance of log-probabilities within each edit direction, which is $O(|Q|)$ per iteration. Is this sufficient, or do off-diagonal terms matter?*

9 Summary

Application	Theoretical basis	Effort	Payoff
Derivative pre-filtering (S1)	McBride derivative $\approx -\Delta L$	Low	High
Synergy detection (S2)	Emergent synergy $\sigma_{y,z} > e$	Medium	Medium-High
Beam-width damping (S3)	Delay-logistic stability criterion	Medium	Medium
Statistics smoothing (S3)	Delayed-feedback dynamics	Medium	Medium
Natural gradient (S4)	Fisher metric, WT §21.4	High	Uncertain
ODE convergence monitor (S5)	Quantale exponential/logistic ODE	Low	Medium

The McBride-derivative analysis of CLA is not merely a formal exercise. It yields concrete engineering guidance: a cheap proxy for the expensive MDL evaluation, a principled detector for joint proposals that greedy search misses, a dynamical-systems framework for diagnosing and fixing convergence pathologies, and a geometry-aware search direction that may improve efficiency on redundant streams. The integration roadmap provides a path from the current **chaoslang** prototype to a McBride-aware implementation, one independent step at a time.

References

- [1] ZeroBot working notes for Ben Goertzel, *Bridging the Chaos Language Algorithm and McBride-Derivative Pattern Calculus: One-Hole Contexts for Grammar Induction*, Note 0007, 2026-07-05.

- [2] Ben Goertzel w/ GPT-5.5-Pro, *The Chaos Language Algorithm: An Implementation-Oriented Specification*, 2026-07-03.
- [3] Ben Goertzel and Collaborators, *A Hyperon-Ready Python Architecture for the Chaos Language Algorithm*, 2026-07-03.
- [4] Ben Goertzel, *Weakness is All You Need*, version 10, 2025-12-15. Chapters 21–22.
- [5] `chaoslang` prototype repository, `projects/chaos-language-algorithm/repos/chaoslang/`.